



به نام خدا

موضوع: گزارش کار پروژه ساختمان داده و الگوریتم

(BasuKala)

درس: ساختمان داده و الگوریتم

استاد: جناب آقای دکتر علی جاویدانی

اعضای تیم: نورا پناهی¹, مینا زرافشان²

شماره دانشجویی: 40312358016², 40332999086¹

ترم ۴۰۴۱

فهرست مطالب

4	مقدمه
4	اهداف پروژه
5	معماری و کلاس‌های کلیدی
5	لایه Domain
5	Basket
6	City
6	Order
6	Product
6	User
7	لایه Services
7	PurchaseService
7	PurchaseHistory
8	DeliveryService
8	ProductService
9	لایه Structure
9	Graph
9	BST
9	ProductMaxHeap
10	OrderPriorityQueue
10	UserHashTable
10	ItemStack
11	Trie

12	Manage لایه
12	BasuKala
14	بخش ترمیمی
17	منابع

مقدمه

در این پروژه یک سیستم فروشگاه آنلاین با استفاده از زبان برنامه‌نویسی C++ طراحی و پیاده‌سازی شده است. هدف اصلی پروژه، ترکیب مفاهیم برنامه‌نویسی شی‌گرا (OOP) با ساختارهای داده مختلف در قالب یک سیستم کاربردی بوده است.

این سیستم امکان ثبت‌نام و ورود کاربران، مدیریت محصولات، استفاده از سبد خرید، ثبت سفارش و پردازش ارسال سفارش‌ها را فراهم می‌کند. همچنین برای بهینه‌سازی بخش‌های مختلف برنامه از ساختارهای داده‌ای مانند درخت جستجوی دودویی (BST)، 'LinkedList'، 'Trie'، 'Heap'، 'Graph' و 'Stack' استفاده شده است.

اهداف پروژه

هدف این پروژه، طراحی و پیاده‌سازی یک سیستم فروشگاه آنلاین با تمرکز بر به‌کارگیری عملی مفاهیم برنامه‌نویسی شی‌گرا و ساختمان داده‌ها در یک مسئله واقعی بوده است.

از مهمترین اهداف پروژه می‌توان به موارد زیر اشاره کرد:

- طراحی یک سیستم ماژولار با تفکیک مناسب مسئولیت‌ها
- پیاده‌سازی نقش‌های مختلف کاربری (کاربر عادی و ادمین)
- مدیریت محصولات، سبد خرید و ثبت سفارش به صورت ساخت‌یافته
- استفاده عملی از ساختمان داده‌هایی مانند BST، 'Trie'، 'LinkedList'، 'Heap'، 'Graph' و 'Stack'
- افزایش کارایی عملیات‌هایی مانند جستجو، مرتب‌سازی، پردازش سفارش و مسیریابی ارسال

در این پروژه تلاش شده است تا مفاهیم تئوری ساختمان داده و طراحی نرم‌افزار در قالب یک سیستم کاربردی و قابل توسعه پیاده‌سازی شوند.

معماری و کلاس‌های کلیدی

در این پروژه ساختار سیستم به صورت لایه‌بندی شده طراحی شده است تا تفکیک مسئولیت‌ها رعایت شود و هر بخش وظیفه مشخصی داشته باشد. معماری کلی پروژه شامل چهار لایه اصلی است:

لایه Domain

این لایه شامل موجودیت‌های اصلی سیستم بوده و نمایانگر مدل مفهومی فروشگاه می‌باشد. کلاس‌های این بخش مسئول نگهداری داده‌های پایه سیستم هستند و تمرکز آن‌ها بر تعریف ساختار اشیا است، نه پیاده‌سازی منطق‌های پیچیده اجرایی.

نمونه کلاس‌ها:

- Basket.h
- User.h
- City.h
- Order.h
- Product.h

وظیفه اصلی این لایه، تعریف ساختار داده‌ای اشیای اصلی سیستم و مشخص کردن ویژگی‌های آن‌هاست. اکثر کلاس‌های این بخش دارای پارامتری به نام id هستند که به عنوان شناسه یکتا برای هر شیء استفاده می‌شود. این شناسه نقش مهمی در مدیریت، جستجو، دسته‌بندی و پردازش داده‌ها در بخش‌های مختلف برنامه ایفا می‌کند.

کلاس Basket:

- **هدف:** این کلاس، سبد خرید یک کاربر را نمایش می‌دهد و امکان افزودن، حذف و مشاهده محصولات موجود در آن را فراهم می‌کند.

• متدهای اصلی:

- `addProduct(const Product& product)`: یک محصول را به سبد خرید اضافه می‌کند. همچنین قیمت محصول به مجموع قیمت سبد خرید (`totalPrice`) اضافه می‌شود.
- `removeProductByIndex(int id)`: محصولی با شناسه `id` را از سبد خرید حذف می‌کند. اگر محصول یافت شود، قیمت آن از مجموع قیمت سبد خرید کم شده و محصول به پشته `removedStack` منتقل می‌شود تا امکان بازگرداندن آن وجود داشته باشد.
- `undoLastRemovedItem()`: آخرین محصول حذف‌شده از سبد خرید را از پشته `removedStack` برمی‌گرداند و به سبد خرید اضافه می‌کند. قیمت محصول به مجموع قیمت سبد خرید اضافه می‌شود.

کلاس City:

- **هدف:** این کلاس، اطلاعات مربوط به یک شهر را نگهداری می‌کند.
- **متدهای اصلی:**
- `hasWareHouse()`const: بررسی می‌کند که شهر دارای انبار است یا خیر.

کلاس Order:

- **هدف:** این کلاس، اطلاعات مربوط به یک سفارش را نگهداری می‌کند.
- **متدهای اصلی:**
- توابع `get...`: وظیفه دسترسی و بازیابی اطلاعات مربوط به یک سفارش خاص را دارند. به عبارت دیگر، این توابع به شما اجازه می‌دهند تا بدون تغییر دادن اطلاعات سفارش، مقادیر مختلف آن را مشاهده کنید.
- `setRoute(...)`: مسیر تحویل سفارش (`deliveryPath`)، فاصله (`distance`) و شناسه انبار مبدأ (`warehouseId`) را تنظیم می‌کند.

کلاس Product:

- **هدف:** این کلاس، اطلاعات مربوط به یک محصول را نگهداری می‌کند.
- **متدهای اصلی:**
- توابع `get...`: در هر یک از این توابع، متغیرهایی مانند نام، قیمت، شناسه محصول، شناسه دسته و تعداد فروش هر محصول برگردانده می‌شود.
- `increaseSoldCount()`: تعداد فروش محصول را یک واحد افزایش می‌دهد.

کلاس User:

- **هدف:** این کلاس، اطلاعات مربوط به یک کاربر را نگهداری می‌کند.
- **متدهای اصلی:**
- `getHistory()`: یک ارجاع به شی `PurchaseHistory` را برمی‌گرداند (برای دسترسی به تاریخچه خرید کاربر).
- `increase/decreaseBalance(int amount)`: موجودی حساب کاربر را به میزان `amount` افزایش/کاهش می‌دهد.
- `hasEnoughBalance(int amount)const`: بررسی می‌کند که کاربر مجودی کافی برای پرداخت مبلغ `amount` را دارد یا خیر.

لایه Services

PurchaseService

- کلاس PurchaseService هسته اصلی مدیریت فرآیند خرید در سیستم است و مسئول کنترل کاربران، سبد خرید و ثبت سفارش‌ها می‌باشد. این کلاس نقش واسطه بین مدل‌های دامنه مانند User ، Product ، Order و سایر بخش‌های اجرایی سیستم را ایفا می‌کند.
- این کلاس با استفاده از شناسه‌های یکتا برای کاربران و سفارش‌ها، مدیریت دقیق داده‌ها را تضمین کرده و با به‌روزرسانی همزمان آمار فروش محصولات، هماهنگی بین بخش‌های مختلف سیستم را حفظ می‌کند.
- به‌طور کلی، PurchaseService، امکاناتی را فراهم می‌آورد که شامل موارد زیر است:
- **مدیریت کاربران:** از طریق متد registerUser امکان ثبت‌نام کاربران جدید با تعیین نقش، نام، موجودی اولیه را فراهم می‌کند. متد login امکان ورود کاربران به سیستم را با اعتبارسنجی نام کاربری فراهم می‌کند و متد logout امکان خروج کاربران را فراهم می‌سازد. همچنین، متدهای user_getter و userExists امکان دسترسی و بررسی وجود کاربران را براساس شناسه یا نام فراهم می‌کنند.
 - **مدیریت سبد خرید:** متدهای addToBasket و removeFromBasket امکان افزودن و حذف محصولات از سبد خرید کاربر فعلی را فراهم می‌کنند. متد checkIfBasketExists وضعیت سبد خرید را بررسی می‌کند.
 - **نهایی‌سازی سفارش:** متد checkout فرآیند نهایی‌سازی خرید را مدیریت می‌کند. این متد شامل بررسی ورود کاربر، خالی نبودن سبد خرید، موجودی کافی حساب کاربر، کسر مبلغ سفارش از حساب کاربر، ایجاد سفارش جدید، ثبت سفارش در تاریخچه خرید کاربر و پاک کردن سبد خرید است.
 - **مدیریت تاریخچه خرید:** متد showPurchaseHistory امکان نمایش تاریخچه خرید کاربر فعلی را فراهم می‌کند.
 - **مدیریت داده‌ها:** متد showAllUsers امکان نمایش لیست تمام کاربران ثبت شده را فراهم می‌کند.

PurchaseHistory

- کلاس PurchaseHistory مسئول نگهداری و مدیریت تاریخچه خرید هر کاربر در سیستم است. این کلاس سفارش‌های ثبت‌شده را به‌صورت یک ساختار لیست پیوندی (Linked List) ذخیره می‌کند تا امکان افزودن سفارش‌های جدید به انتهای لیست و پیمایش آن‌ها به ترتیب ثبت فراهم باشد.
- در این کلاس از ساختار داده‌ای Node (که یک struct داخلی است) استفاده شده هر گره از لیست پیوندی را نشان می‌دهد. هر گره شامل یک شیء از نوع Order و اشاره‌گری به گره بعدی (next) است. با هر بار نهایی‌سازی خرید، سفارش جدید به انتهای لیست افزوده می‌شود. همچنین در صورت نیاز، امکان نمایش کامل تاریخچه خرید کاربر از طریق پیمایش لیست وجود دارد.
- در سازنده مخرب (Destructor)، حافظه اختصاص داده‌شده به گره‌ها به‌صورت کامل آزاد می‌شود تا از نشت حافظه جلوگیری گردد. این موضوع نشان‌دهنده مدیریت صحیح منابع در طراحی کلاس است.
- متدهای اصلی این کلاس شامل موارد زیر است:
- **addOrder(const Order& order):** یک order جدید را به انتهای لیست تاریخچه خرید اضافه می‌کند. اگر لیست خالی باشد، head و tail هر دو به گره جدید اشاره می‌کنند. در غیر این صورت، گره جدید به انتهای لیست اضافه شده و tail به‌روزرسانی می‌شود.
 - **display(PService& pservice) const:** این متد با پیمایش لیست از head تا tail، اطلاعات هر سفارش (شامل شناسه سفارش، مبلغ کل و لیست محصولات) را نمایش می‌دهد. برای نمایش اطلاعات محصولات، از کلاس PService استفاده شده است.

DeliveryService

کلاس DeliveryService مسئول مدیریت و پردازش سفارش‌های ثبت‌شده و انجام فرآیند ارسال آن‌ها است. این کلاس سفارش‌ها را در یک صف اولویت‌دار (Priority Queue) به نام OPQ نگهداری می‌کند تا پردازش آن‌ها بر اساس معیار مشخصی (مانند زمان ثبت یا اولویت) انجام شود.

متد addOrder با ثبت هر سفارش، شی مربوطه را به صف (OPQ) اضافه می‌کند. در متد dispatchNext، سفارش از صف خارج شده و با استفاده از ساختار گراف، مسیر بهینه ارسال (با استفاده از متد getDeliveryRoute) از انبار تا شهر مقصد محاسبه می‌گردد. اطلاعات مربوط به انبار انتخاب‌شده، فاصله و مسیر طی‌شده در شیء سفارش ذخیره می‌شود.

در صورتی که ارسال موفق باشد، امتیاز کاربر بر اساس تعداد محصولات خریداری‌شده افزایش می‌یابد (متد increaseScore از کلاس User). همچنین اطلاعات کامل ارسال برای نمایش یا ثبت گزارش چاپ می‌شود. این طراحی باعث جداسازی کامل منطق ارسال از فرآیند خرید شده و اصل تفکیک مسئولیت‌ها را رعایت می‌کند.

ProductService

کلاس PService مسئول مدیریت کامل محصولات در سیستم است و به‌عنوان مرکز کنترل داده‌های مربوط به کالاها عمل می‌کند. این کلاس وظیفه افزودن، حذف، جستجو و به‌روزرسانی اطلاعات محصولات را بر عهده دارد و برای افزایش کارایی، از چندین ساختمان داده به‌صورت هم‌زمان استفاده می‌کند.

در این کلاس، هر محصول دارای یک شناسه یکتا (ProductId) است که در یک ساختار unordered_map ذخیره می‌شود که شناسه محصول را به عنوان کلید و یک اشاره‌گر unique_ptr به شی Product را به عنوان مقدار نگهداری می‌کند. استفاده از unique_ptr مدیریت حافظه را خودکار می‌کند و از نشت حافظه جلوگیری می‌کند. علاوه بر این، محصولات در درخت‌های جستجوی دودویی (BST) بر اساس دسته‌بندی ذخیره می‌شوند تا امکان نمایش مرتب آن‌ها فراهم شود. هر درخت BST مربوط به یک دسته‌بندی از محصولات است.

برای جستجوی سریع بر اساس نام یا پیشوند نام محصول، از ساختار Trie استفاده شده است. همچنین جهت نگهداری و به‌روزرسانی پر فروش‌ترین کالا، یک Heap بیشینه (Max Heap) به کار رفته است که پس از هر خرید، مقدار فروش محصول را به‌روزرسانی می‌کند.

این طراحی باعث شده عملیات‌هایی مانند جستجو، حذف، دسته‌بندی و محاسبه پر فروش‌ترین کالا با کارایی بالا و پیچیدگی زمانی مناسب انجام شوند.

لایه Structure

Graph

این کلاس مسئول نگه داری شهر ها و نیز همسایه های هر یک از آنهاست. به این صورت که هر شهر دارای یک آیدی یکتا می باشد که در کلاس گراف با کمک پارامتر nextId هنگام ساخت هر شهر بصورت خودکار تولید می شود. با کمک این آیدی ها و یک vector دوبعدی (neighbor) که در آن یک pair از دو int وجود دارد همسایه های هر شهر مشخص می شود. شایان ذکر است که index هر خانه آیدی آن شهر است و در vector آن، مقدار اولیه آیدی همسایه آن خانه و مقدار دوم وزن یال بین دو شهر است.

همچنین یکی از مهم ترین وظایف این کلاس یافتن نزدیکترین انبار به شهر مبدا و انتخاب کوتاهترین مسیر بین مبدا و مقصد است که روند آن به این صورت است که نخست با استفاده از تابع dijkstra کوتاه ترین مسیر بین مبدا و تمامی انبارها پیدا می شود سپس انباری که فاصله کمتری دارد با استفاده از متد nearestWarehouse مشخص می شود. سپس با استفاده از تابعی دیگر مسیر بین دو مکان پیدا می شود. از دیگر وظایف این کلاس میتوان به افزودن شهر و یال بین دو شهر اشاره کرد که به درستی پیاده سازی شده است.

BST

در این کلاس یک درخت جستجوی دودویی (BST) پیاده سازی شده است که برای نگهداری محصولات طراحی شده است. این کلاس دو درخت مجزا دارد: یکی براساس نام محصول (root) و دیگری براساس قیمت محصول (root2). این امکان را فراهم می کند که محصولات را هم براساس نام و هم براساس قیمت جستجو و مدیریت کرد. از جمله متدهای اصلی این کلاس می توان به موارد زیر اشاره کرد:

- insertbyname(const string& key, Product* value): این متد یک محصول جدید را براساس نام در درخت BST ذخیره می کند. اگر درخت خالی باشد، یک گره جدید ایجاد کرده و آن را به عنوان ریشه قرار می دهد. در غیر اینصورت، در درخت جستجو می کند تا مکان مناسب برای درج گره جدید را پیدا کند.
- insertbyprice(const string& key, Product* value): مشابه متد قبلی اما براساس قیمت محصول جستجو و درج انجام می شود.
- remove(const string& key, int productId, Product* value): یک محصول با شناسه مشخص شده را از درخت حذف می کند. در این تابع از تابع کمکی removeHelper استفاده شده است که بصورت بازگشتی در درخت جستجو می کند تا گره موردنظر را پیدا کند.

ProductMaxHeap

کلاس ProductMaxHeap با هدف نگهداری پویا و کارآمد محصولات براساس میزان فروش به عنوان یک Max Heap پیاده سازی شده است. کلاس ذکر شده دارای متدهای اصلی از جمله insert, remove, increaseKey است. توابعی همچون heapifyUp و heapifyDown از جمله توابع کمکی هستند که بعد از حذف یا درج محصول فراخوانده می شوند. همچنین بعد از به فروش رسیدن یک محصول و افزایش امتیاز آن از تابع heapifyUp برای مرتب کردن دوباره هیپ استفاده می شود.

تابع extractMax نیز پرفروش ترین محصول را از هیپ برمیگرداند و آن را از کانتینر حذف می کند. شایان ذکر است از کانتینر vector برای پیاده سازی max heap استفاده شده است.

OrderPriorityQueue

برای نگهداری هر order ، این صف سفارش‌ها را براساس دو معیار مرتب می‌کند:

- امتیاز فریز شده (FrozenScore): سفارش‌هایی با امتیاز فریز شده بالاتر، اولویت بیشتری دارند.
- زمان ثبت (Timestamp): در صورتی که دو سفارش دارای امتیاز فریز شده یکسان باشند، سفارشی که زودتر ثبت شده، اولویت بیشتری دارد.

به منظور برآورده شدن این امر تابع comparator نوشته شد تا با استفاده از آن در متد کمکی heapifyUp تعیین اولویت بین دو محصول صورت گیرد و اضافه کردن order جدید به درستی انجام شود. این کلاس همچنین همانند کلاس ProductMaxHeap، دارای متد حذف به کمک تابع کمکی heapifyDown است.

UserHashTable

این کلاس یک Hash Table را پیاده‌سازی می‌کند که برای ذخیره و مدیریت اطلاعات کاربران (User) طراحی شده است. این سیستم از separate chaining برای رفع collisions استفاده می‌کند. از مهمترین متدهای این کلاس می‌توان به موارد زیر اشاره کرد:

- hashFunction(string& key): یک تابع هش که یک کلید رشته‌ای (نام کاربر) را به یک index در وکتور table تبدیل می‌کند. این تابع از یک الگوریتم ساده ضرب و جمع برای محاسبه مقدار هش استفاده می‌کند.
- insert(const string& key): یک کاربر جدید را به هش اضافه می‌کند. این کار ابتدا با محاسبه کردن index مربوط به نام کاربر با استفاده از تابع hashFunction انجام می‌شود. سپس در زنجیره مربوط به آن index جستجو می‌کند تا ببیند آیا کاربری با همان نام از قبل وجود دارد یا خیر. اگر کاربری با همان نام وجود نداشته باشد، یک گره جدید ایجاد کرده و کاربر را در آن ذخیره می‌کند. سپس گره جدید را به ابتدای زنجیره اضافه می‌کند.
- findByName(const string& name): یک کاربر با نام مشخص شده را در جدول هش جستجو می‌کند. ابتدا، index مربوط به نام کاربر را با استفاده از تابع hashFunction محاسبه می‌کند. سپس در زنجیره مربوط به آن index، جستجو می‌کند تا ببیند آیا کاربری با همان نام وجود دارد یا خیر.

ItemStack

در این کلاس یک پشته (Stack) از نوع LIFO پیاده‌سازی شده است که برای ذخیره اشیاء Product در قالب محصولات حذف شده طراحی شده است. این پشته از یک لیست پیوندی برای ذخیره عناصر استفاده می‌کند. از جمله توابع پیاده‌سازی شده در این کلاس می‌توان به موارد زیر اشاره کرد:

- عملیات push: یک محصول جدید را به بالای پشته اضافه می‌کند. ابتدا یک گره جدید ایجاد می‌کند و محصول را در آن ذخیره می‌کند. اشاره‌گر next گره جدید را به گره فعلی بالای پشته (node) تنظیم می‌کند و در آخر اشاره‌گر node را به گره جدید انتساب می‌دهد.
- عملیات pop: محصول بالای پشته را حذف می‌کند و در متغیر removedItem قرار می‌دهد. در صورت خالی نبودن پشته، اشاره‌گر موقت (temp) به گره بالای پشته (node) تنظیم می‌شود. سپس داده گره بالای پشته در removedItem کپی می‌شود. اشاره‌گر node به گره بعدی انتساب داده می‌شود (به این ترتیب گره بالای پشته حذف می‌شود).

Trie

این کلاس برای ذخیره و جستجوی رشته‌ها طراحی شده است و در کلاس‌های PService و BasuKala مورد استفاده قرار می‌گیرد. همچنین از ساختار درختی برای ذخیره رشته‌ها استفاده شده است به طوری که هر گره نشان‌دهنده یک کاراکتر از رشته است.

از مهمترین توابع این کلاس می‌توان به insert، searchByPrefix و collectAllWords اشاره کرد.

- در تابع insert، یک کلمه جدید به درخت اضافه می‌شود. ابتدا از گره ریشه شروع می‌کند. برای هر کاراکتر در کلمه، index مربوط به آن کاراکتر را محاسبه می‌کند. اگر گره فرزند مربوط به آن index وجود نداشته باشد، یک گره جدید در ایجاد می‌کند و آن را به عنوان گره فرزند تنظیم می‌کند. سپس به گره فرزند جدید می‌رود و این فرآیند را برای کاراکتر بعدی تکرار می‌کند. پس از پردازش تمام کاراکترهای کلمه، مقدار endOfTheWord گره فعلی را true تنظیم می‌کند و کلمه را در name ذخیره می‌کند.
- در تابع searchByPrefix، تمام کلماتی را که با پیشوند مشخص شده شروع می‌شوند، پیدا می‌کند. از گره ریشه شروع می‌کند. برای هر کاراکتر، index مربوط به آن کاراکتر را محاسبه می‌کند. اگر گره فرزند مربوط به آن index وجود نداشته باشد، یک vector خالی را برمی‌گرداند (به این معنی که هیچ کلمه‌ای با پیشوند مشخص شده وجود ندارد). سپس به گره فرزند جدید می‌رود و این فرآیند را برای کاراکتر بعدی تکرار می‌کند. پس از پردازش تمام کاراکترهای پیشوند، تابع collectAllWords را فراخوانی می‌کند تا تمام کلمات موجود در زیردرخت فعلی را جمع‌آوری کند. در نهایت، vector ای از کلمات جمع‌آوری شده را برمی‌گرداند.
- تابع collectAllWords، یک تابع بازگشتی است که تمام کلمات موجود در زیردرخت با ریشه node را جمع‌آوری می‌کند. اگر مقدار endOfTheWord گره فعلی true باشد، کلمه مربوط به آن گره را به results اضافه می‌کند. سپس برای هر گره فرزند، تابع collectAllWords به صورت بازگشتی فراخوانی می‌شود.

لایه Manage

BasuKala

در این کلاس سیستم فروشگاه‌های یا مدیریت سبد خرید را مدل‌سازی می‌کند. این کلاس شامل ویژگی‌ها و متدهایی برای مدیریت کاربران، محصولات، سبد خرید، سفارشات و فرآیند تحویل است.

۱. اعضای کلاس

- **Purchase**: یک شی از نوع PurchaseService که بطور کلی مسئول مدیریت کاربران، ورود/خروج، سبد خرید و تاریخچه خرید است.
- **pservice**: یک شی از نوع PService که مسئول مدیریت محصولات، دسته‌بندی‌ها و جستجو است.
- **graph**: یک شی از نوع Graph که برای مدل‌سازی شهرها و مسیرهای تحویل استفاده می‌شود.
- **delivery**: یک شی از نوع DeliveryService که مسئول مدیریت و ارسال سفارشات است.

۲. کانستراکتور

در سازنده این کلاس، داده‌های اولیه مانند کاربران، محصولات، شهرها و مسیرهای ارتباطی بین شهرها ایجاد و مقداردهی اولیه می‌شوند. همچنین چند کاربر با نقش‌های مختلف (normal و admin) برای بخش login تعریف شده‌اند. محصولات مختلف با نام، قیمت و دسته‌بندی‌های مختلف به سیستم اضافه شده‌اند. همچنین شهرها و مسیرهای ارتباطی بین آنها نیز تعریف شده است.

۳. متدهای اصلی

- **run()**: این تابع فرآیند سیستم را شروع می‌کند. در هر تکرار حلقه اصلی برنامه، براساس انتخاب و نوع کاربر، متدهای مربوطه فراخوانی می‌شود.
- **firstPage()**: این متد صفحه ابتدایی برنامه را نمایش می‌دهد و به کاربر این امکان را می‌دهد تا بین sign، up، login و exit یکی را انتخاب کند.
- **signUp()**: این متد به کاربر اجازه می‌دهد تا یک حساب کاربری جدید ایجاد کند. ابتدا کاربر نام خود را وارد کرده سپس سیستم با متد userExists بررسی می‌کند که آیا کاربری با این نام در سیستم وجود دارد یا خیر. اگر کاربر با این نام وجود نداشته باشد، یک کاربر جدید با نقش normal و موجودی اولیه 0، توسط تابع registerUser ساخته می‌شود. در آخر کاربر با استفاده از متد login (از کلاس PurchaseService)، وارد حساب کاربری خود می‌شود.
- **login()**: این متد به کاربر اجازه می‌دهد تا با استفاده از نام کاربری خود وارد سیستم شود. همانند تابع قبلی سیستم وجود کاربر را بررسی می‌کند و در اینجا اگر کاربر وجود داشته باشد، کاربر را وارد سیستم می‌کند (با استفاده از متد login از کلاس PurchaseService).

۴. متدهای منوی ادمین

- اگر کاربر وارد شده ادمین باشد، منوی مخصوص او فراخوانی می‌شود (adminMenu).
- در تابع adminMenu، این امکان را به ادمین می‌دهد تا کالایی در سیستم اضافه یا حذف کند، اطلاعات کاربران عادی را مشاهده و سفارشات را تحویل دهد. هرکدام از این قابلیت‌ها در توابع زیر پیاده شده است.
- **addProductAdmin()**: این متد به ادمین این اجازه را می‌دهد تا با وارد کردن نام، قیمت و آیدی دسته کالایی به سیستم اضافه کند. این عمل توسط شی pservice، با استفاده از تابع addProduct امکان پذیر می‌باشد.
- **removeProductAdmin()**: در این تابع ابتدا لیست دسته‌بندی‌های محصولات توسط شی pservice به ادمین نشان داده می‌شود تا ادمین بتواند دسته‌بندی مربوط به محصولی که می‌خواهد حذف کند را انتخاب کند. سپس لیستی از محصولات موجود در آن دسته توسط شی bst از کلاس BST، به ادمین نشان داده می‌شود و از ادمین

- خواسته می‌شود تا آیدی محصولی را که می‌خواهد حذف کند را وارد کند. اگر تمام اعتبارسنجی‌ها با موفقیت انجام شوند، محصول را با استفاده از متد `removeProduct` در کلاس `PService` از سیستم حذف می‌کند.
- `deliverOrders()`: این متد به ادمین اجازه می‌دهد تا سفارشات آماده تحویل را برای ارسال به مشتریان واگذار کند. ابتدا با استفاده از متد `hasOrders` بررسی می‌شود که سفارشی وجود داشته باشد و سپس تا زمانی که سفارشی در صف تحویل وجود داشته باشد، یک حلقه اجرا می‌شود. در هر تکرار این حلقه، سفارش بعدی را از صف تحویل دریافت می‌کند. تمام عملیات حمل و نقل سفارشات در تابع `dispatchNext` در کلاس `DelivaryService` انجام پذیر است.
- `Logout()`: این متد به همراه تابع کمکی `logout` از شی `Purchase`، کاربر را از سیستم خارج می‌کند.

۵. متدهای منوی کاربر عادی

- اگر کاربر وارد شده کاربر عادی باشد، منوی مخصوص او فراخوانی می‌شود.
- در تابع `normalMenu`، منوی اصلی به کاربر نشان داده می‌شود و در حلقه `while`، دو تابع `secondPageNormal()` و `storePageNormal()` براساس خواسته کاربر فراخوانی می‌شود.
- `secondPageNormal()`: در این تابع ابتدا مشخصات کاربر را نشان می‌دهد و سپس کاربر می‌تواند از بین گزینه‌های خرید، افزایش موجودی، مشاهده تاریخچه خرید و خروج از حساب یکی را انتخاب کند. همچنین در این صفحه کاربر می‌تواند پر فروش ترین محصول فروشگاه را با استفاده از متد `getBestSellerHeap().top()` مشاهده کند.
- `increaseBalance()`: اگر کاربر گزینه افزایش موجودی را انتخاب کند، این تابع فراخوانی می‌شود. او می‌تواند با وارد کردن مبلغ و سپس فراخوانی تابع `increaseBalance` در کلاس `PurchaseService` باعث افزایش موجودی خود شود.
- `storePageNormal()`: این متد زمانی فراخوانی می‌شود که کاربر گزینه خرید را انتخاب کرده باشد. این متد بطور کلی منوی فروشگاه را نمایش می‌دهد (دسترسی به دسته‌بندی‌ها، جستجو، اصلاح سبد خرید و تکمیل خرید)
- `showCategories()`: این متد دسته‌بندی‌های محصولات را نمایش می‌دهد و به کاربر اجازه می‌دهد تا یک دسته‌بندی را انتخاب کرده سپس محصولات آن را از `BST` مربوطه دریافت کند و مشاهده کند. همچنین این متد امکان جابجایی ترتیب نمایش براساس نام و قیمت را فراهم می‌کند که این امر توسط تابع `printProducts` از شی `bst` انجام می‌پذیرد. همچنین کاربر بعد از مشاهده محصولات با وارد کردن آیدی کالای موردنظر، می‌تواند کالا را به سبد خرید خود اضافه کند.
- `search()`: این متد به کاربر اجازه می‌دهد تا محصول مورد نظر خود را بر اساس نام جستجو کند. ابتدا از کاربر نام محصول موردنظر را می‌خواهد و آن را دریافت می‌کند. سپس از توابع ساختمان داده `Trie` برای جستجوی محصولات با نام مشابه استفاده می‌کند. در آخر نتایج جستجو را نمایش می‌دهد و به کاربر اجازه می‌دهد تا یک محصول را برای افزودن به سبد خرید انتخاب کند.
- `editCart()`: کاربر با انتخاب اصلاح سبد خرید، این متد برای او فراخوانی می‌شود. ابتدا محصولات موجود در سبد خرید را نمایش می‌دهد. سپس از کاربر می‌خواهد تا آیدی محصولی که می‌خواهد حذف کند را وارد کند. اگر آیدی محصول معتبر باشد، با استفاده از تابع `removeFromBasket` از شی `Purchase` محصول را از سبد خرید حذف می‌کند.
- `undolastremoveditem()`: این تابع این امکان را به کاربر می‌دهد تا در صورت پشیمانی از حذف کالاهایی، آخرین کالای حذف شده (در صورت وجود) را به سبد اضافه کند. این تابع از متد `undoLastRemovedItem` در شی `Purchase` و ساختمان داده `Stack` استفاده می‌کند.

- `completePurchase()`: این متد فرآیند تکمیل خرید را انجام می‌دهد و سفارش را ثبت می‌کند. در این تابع در صورت وجود سفارش، لیستی از شهرها را نشان کاربر می‌دهد (متد `getCitiesList` از کلاس `Graph`) و کاربر با وارد کردن آیدی شهر موردنظر خرید خود را تکمیل کرده که این امر توسط دو تابع `checkout` و `addOrder` میسر می‌شود.

بخش ترمیمی

Hash

- نقش اصلی: دسترسی سریع و مستقیم به داده‌ها از طریق کلید، با برخورد با برخورد (collisions) به روش مناسب.
- مزایا: دسترسی $O(1)$ تقریباً برای جستجوهای کاربرها و محصولات براساس کلید (نام کاربری، شناسه محصول و غیره).
- کاربردها در پروژه: نگهداری اطلاعات کاربران، و همچنین امکان دسترسی سریع به داده‌های مرتبط با کاربران.

Trie

- نقش اصلی: ذخیره و جستجوی موثر رشته‌ها با پیشوندهای مشترک.
- مزایا: جستجوی پیشوندی سریع با زمان وابسته به طول رشته؛ امکان `autocomplete` و پیشنهادهای نام کالاها.
- کاربردها در پروژه: جستجوی سریع محصولات با نام یا پیشوند، ارائه نتایج مشابه به کاربر و کمک به عملیات خرید با یافتن سریع کالاهای مرتبط.

Stack

- نقش اصلی: مدل کردن رفتارهای LIFO برای نگهداری آیتم‌های موقتی با امکان `undo`.
- مزایا: بازیابی ساده‌ترین شکل بازگشت به وضعیت قبل با حذف آخرین مورد و امکان بازگرداندن آن (`undo`).
- کاربردها در پروژه: نگهداری محصولاتی که اخیراً از سبد خرید حذف شده‌اند تا امکان بازگردانی سریع وجود داشته باشد.

LinkedList

- نقش اصلی: نگهداری پیوسته داده‌ها با امکان پیمایش ساده و درج/حذف انتهای لیست، معمولاً بدون دسترسی تصادفی به عنصر خاص.
- مزایا: پیوستگی ساده، حافظه کمتر نسبت به آرایه‌های بزرگ، امکان پیوسته کردن تاریخچه یا مجموعه‌ای از سفارش‌ها با هزینه پایین.
- کاربردها در پروژه: تاریخچه خرید کاربر (`PurchaseHistory`) را به صورت `Linked List` نگهداری می‌کند تا اضافه کردن سفارش جدید به انتهای تاریخچه ساده باشد و پیمایش انجام شود.

Graph

- نقش اصلی: مدل‌سازی شبکه شهری با شهرها به عنوان گره‌ها و ارتباطات بین آنها به عنوان یال‌ها با وزن مشخص (مسیر/فاصله).
- مزایا: یافتن مسیر بهینه بین مبدا و مقصد، تخمین هزینه و زمان تحویل و پشتیبانی از محاسبه مسیرهای مختلف.
- کاربردها در پروژه: محاسبه مسیر تحویل بین انبارها و شهرهای مقصد؛ تعیین نزدیک‌ترین انبار و مسیر بهینه با استفاده از الگوریتم‌های کوتاه‌ترین مسیر.

BST

- نقش اصلی: نگهداری و جستجوی کارآمد عناصر بر اساس کلیدها (نام و قیمت محصول، هرکدام در یک درخت جداگانه).
- مزایا: جستجو، درج و حذف با زمان نسبتاً $\log(n)$ در اغلب سناریوها؛ نمایش مرتب دسته‌بندی‌شده از کالاها (بر اساس نام یا قیمت).
- کاربردها در پروژه: دو BST مجزا برای هر دسته از محصولات (یکی بر پایه نام و دیگری بر پایه قیمت) که امکان جستجو، افزودن و حذف کارآمد کالاها را فراهم می‌کند.

Max Heap

- نقش اصلی: نگهداری عناصر با اولویت بالا به صورت ساختار heap برخلاف لیست‌ها یا آرایه‌های ساده.
- مزایا: استخراج سریع بزرگترین مقدار (پرفروش‌ترین کالا یا سفارش با اولویت بالا)، بروزرسانی اولویت‌ها به شکل کارآمد.
- کاربردها در پروژه: نگهداری پرفروش‌ترین کالاها و مدیریت دسته‌های اولویت‌بندی‌شده.

Priority Queue

- نقش اصلی: نگهداشت سفارش‌ها یا کالاها بر پایه اولویت مشخص؛ امکان استخراج سریع با بالاترین اولویت.
- مزایا: تضمین تقدم‌دادن به آیتم‌های با اولویت بالاتر، پیاده‌سازی منطق زمان ثبت و امتیاز کاربر برای تصمیم‌گیری‌های تحویل.
- کاربردها در پروژه: مدیریت سفارش‌ها و تسهیل فرآیند تحویل به وسیله صف‌های اولویت‌دار؛ سازوکارهای مشابه برای پردازش سفارشات با امتیازهای متفاوت.

معرفی ساختمان داده Red-Black Tree

- Red-Black Tree یک درخت دودویی متوازن با قوانین رنگی است: گره‌ها قرمز یا سیاه‌اند، ریشه سیاه است، برگ‌های NIL سیاه‌اند، هیچ گره قرمزی پی در پی نیست و هر مسیر به برگ‌ها مقدار معینی از گره‌های سیاه دارد.
- BST ساده هیچ قاعده‌ای برای تعادل ندارد؛ با ورود داده‌های نامنظم، در بدترین حالت ممکن است به شکل درختی نامتوازن و عمق $O(n)$ برسد.
- نتیجه کلیدی Red-Black Tree: همواره ارتفاع درخت را به طور تقریبی $O(\log n)$ نگه می‌دارد؛ BST تنها در حالت ایده‌آل چنین تعادلی ندارد.

عملیات درج در Red-Black Tree

- روند کلی: مشابه درج در BST است (یعنی جای مناسب برای کلید پیدا می‌شود و گره اضافه می‌شود)، اما گره تازه اضافه شده به طور موقت قرمز در نظر گرفته می‌شود.
- نقطه حساس: اضافه شدن گره قرمز ممکن است قوانین Red-Black را نقض کند. برای اصلاح از ترکیبی از رنگ‌آمیزی و چرخش‌های راست/چپ استفاده می‌شود تا قوانین Red-Black دوباره برقرار شوند.
- نتیجه عملی: پس از درج، درخت در حالت تعادل باقی می‌ماند و عمق درخت نسبتاً کوچک باقی می‌ماند.
- اهمیت در پروژه: درج با Red-Black Tree زمان جستجو و دسترسی را در مقیاس‌های پایدار و داده‌های پویا حفظ می‌کند، به ویژه در سیستم‌هایی که مرتب سازی و جستجو بر پایه کلیدهای گوناگون وجود دارد.

عملیات حذف در Red-Black Tree

- روند کلی: حذف گره با همانند BST انجام می‌شود؛ در صورت وجود دو فرزند از جایگزینی با successor/predecessor استفاده می‌شود.
- نکته کلیدی: پس از حذف، ممکن است نقض قوانین Red-Black پیش آید. با مجموعه‌ای از حالات بازنویسی رنگ و چرخش‌ها دوباره تعادل برقرار می‌شود.
- نتیجه عملی: حذف در Red-Black Tree با وجود پیچیدگی بیشتر نسبت به BST ساده، با حفظ تعادل، زمان‌های جستجو و عملیات مرتبط را به صورت پایدار نگه می‌دارد (حدوداً $O(\log n)$).
- اهمیت در پروژه: حذف ایمن و سریع کالاها یا دسته‌بندی‌ها با حفظ کارایی جستجوها و عملیات مرتبط.

مزایای کلیدی Red-Black Tree برای پروژه

- پایداری زمان: هر دو عملیات درج و حذف به طور نسبتاً ثابت در $O(\log n)$ انجام می‌شوند، حتی با ورودی‌های نامنظم.
- حفظ تعادل بلندمدت: به طور مداوم ارتفاع درخت را کنترل می‌کند تا از حالت‌های بد BST جلوگیری شود.
- مناسب برای داده‌های پویا: در فروشگاه با افزودن/حذف مداوم کالاها یا دسته‌ها، Red-Black Tree کارایی قابل اعتمادی فراهم می‌کند.
- ادغام با سایر ساختارها: می‌تواند همراه با BST یا سایر ساختارها استفاده شود تا جستجو و مرتب‌سازی را در سطوح مختلف بهبود دهد (مثلاً نگهداری کالاها بر اساس نام یا قیمت به صورت درخت‌های جداگانه با تعادل مناسب).
- انتخاب Red-Black Tree به جای BST ساده در پروژه به خاطر پایداری زمان و حفظ تعادل در برابر ورودی‌های ناهمگون بسیار منطقی است. این امر به بهبود کارایی جستجو، درج و حذف، به ویژه در سیستم‌های با داده‌های پویا و با مقیاس بزرگ کمک می‌کند. اگر نیاز باشد، می‌توان Red-Black Tree را به صورت مکمل با BST یا در قالب چند درخت با کاربری‌های متفاوت استفاده کرد تا کارایی کلی سیستم بهینه شود.

منابع

- [نمونه ساختارهای محیط تعاملی ترمینالی \(CLI\)](#)
- [آموزش ساختمان داده Trie](#)
- ChatGPT
- سند رسمی پروژه؛ ارائه شده توسط تیم TA
- مرجع اصلی درس (کتاب Fundamentals of Data Structures in C++)

لازم به ذکر است که پروژه در حال حاضر در مخزن GitHub با آدرس عمومی منتشر نشده است. به دلیل قطعی اینترنت و محدودیت‌های شبکه، دسترسی به مخزن در این زمان ممکن نیست. در نتیجه انتشار عمومی آن به پس از رفع اختلال اینترنت موکول می‌گردد.